

```
import torch
import torch.nn as nn

class ImplicativeMemory(nn.Module):
    def __init__(self, input_dim, hidden_dim,
n_patterns):
        super().__init__()
        self.n_patterns = n_patterns
        self.patterns =
nn.Parameter(torch.randn(n_patterns,
input_dim))
        self.metamemory =
nn.Parameter(torch.randn(n_patterns,
input_dim))
        self.W =
nn.Parameter(torch.randn(input_dim,
hidden_dim))

    def forward(self, x_in):
        # ИМПЛИКАТИВНЫЙ ПОИСК
(подсознание)
        distances =
```

```
torch.cdist(x_in.unsqueeze(0), self.patterns)
    best_match = distances.argmin(dim=1)
# Локализация
    x0_meta =
self.metamemory[best_match] #
Метапамять
    return x0_meta, distances.min().item() #
Сила активации
```